What DP-BST Actually Contributed

An audit of the paper's three claims against its own data

Anar N.

2026-09-07

The three claims

The paper advances three contributions: a decision SAT solver built on component caching instead of clause learning; an evaluation of that solver against five references; and the claim that ordering a hash table's collision buckets as balanced binary search trees beats textbook separate chaining.

Two of the three are restatements of settled results. The genuinely new material is somewhere the paper does not look.

Claim 1: tree-structured hash buckets

This is standard practice rather than a proposal. `java.util.HashMap` has treeified its buckets since Java 8 (2014), converting a bucket to a red-black tree once it exceeds eight entries, ordering that tree primarily by the full hash, and falling back to key comparison only on a hash tie. That is the design in `cache/tree.rs`, including the trick the paper presents as its own reasoning in Section III. It has been shipping in a standard library for a decade.

Worse, the experiment does not measure what it claims to measure. Table III reports mean bucket depth of 1.39 for AVL against 1.60 for chaining, a 13% reduction. For a chain, the mean depth of a successful lookup has a closed form: with `n` entries in `m` buckets and `lambda = n/m`, it is `(lambda + 2) / 2`. Inverting that on the measured data recovers the bucket count to under one percent on every large instance:

instance	entries	measured depth	implied buckets	actual buckets
php-11-10	582,908	2.110	262,504	262,144
aim-100-2_0-no-3	209,929	2.598	65,690	65,536
aim-100-1_6-no-3	17,785	2.098	8,096	8,192

instance	entries	measured depth	implied buckets	actual buckets
bf2670-001	5,345	2.305	2,044	2,048

Bucket depth is a function of the load factor and nothing else. The load factor is `--target-load`, a command line flag defaulting to 4, so the table oscillates between two and four entries per bucket. Table III recovers arithmetic. It contains no information about SAT, about component caches, or about the key distribution, and it would read identically if the table held phone numbers.

Two secondary points survive. The metric counts comparisons on a hit, yet 73% of lookups across the benchmark set are misses, where a chain pays its full length and a tree pays only its height. And the numbers are per-instance unweighted, so small instances dominate; weighting by entries moves AVL against chain from 1.39/1.60 to 1.78/2.21, and maximum depth from 2.8/5.6 to 4.0/11.9. The paper measures the less common case and understates its own effect.

Claim 2: component caching without clause learning

Settled twenty years ago, and the paper concedes as much. Bayardo and Pehoushek introduced it in 2000, Cachet combined it with clause learning in 2004, sharpSAT refined the key representation in 2006. All three targeted model counting, where component caching remains the correct architecture because exhausting the search space is the entire task. For decision SAT the answer arrived early: clause learning dominates, and the two mechanisms conflict, because a learned clause spanning two components merges them.

A PAR-2 of 2.54 against Kissat's 0.50 reproduces that conclusion. Reproduction has value. It is not a contribution.

What is left of the design

One narrow thing, and the paper does not argue for it.

Component caches face a soundness and speed tradeoff on the key. Keys are variable-length byte strings (here, delta-encoded varint index sets), so an exact cache pays a `memcmp` per comparison. The counting community's answer has been to discard the key and keep only its hash, accepting a small probability of a wrong answer; GANAK is explicit that this is what makes it probabilistic.

Hash-ordered tree buckets occupy the middle. Because the low bits of the hash select the bucket, the remaining bits are uniform within it, so the full 64-bit hash works as the tree's *ordering* key rather than as a filter. A comparison is one machine word in essentially every case, and the byte key is touched only on a full 64-bit collision. The result is the per-comparison cost of the lossy design with the exactness of the expensive one, and the same property makes insertion

order random, which is why the unbalanced tree lands at 1.47 against AVL's 1.39 and the balancing machinery earns almost nothing.

Every ingredient is standard. The framing, that this is how an exact component cache buys back the cost that pushed the field toward lossy ones, is the part I have not seen stated, which is a modest claim about presentation.

What the paper found and did not recognise

The strongest result in the project is sitting unremarked in `ablation.csv`.

On `php-11-10`, bounded variable elimination removes ten of the formula's 110 variables and ten of its 561 clauses, a 1.8% reduction. The effect on search:

configuration	nodes	cache entries	hit rate	time
preprocessing on	1,132,265	582,908	48.5%	2.204 s
preprocessing off	8,105	4,097	49.5%	0.028 s

Removing ten variables costs a factor of 140. The effect decides outcomes: `php-12-11` and `php-13-12` both time out at ten seconds with preprocessing enabled and are solved in 0.099 s and 0.263 s with it disabled. The paper's default configuration is two orders of magnitude slower than one flag away, on the family it showcases as its best result.

The paper notices the opposite direction of the same interaction. It reports that dubois is solved almost entirely by preprocessing with the cache contributing nothing, and presents that as an honest null result. It does not check whether the interference runs the other way.

The explanation

The pigeonhole encoding for 11 pigeons and 10 holes is 11 clauses of width 10 (each pigeon occupies some hole) and 550 binary clauses (no two pigeons share a hole). Take a variable `p(1,1)`. It occurs positively in exactly one clause, the width-10 clause for pigeon 1, and negatively in ten binary clauses pairing pigeon 1 with each other pigeon in hole 1. Eliminating it produces `1 x 10 = 10` resolvents replacing 11 clauses, so the clause count falls by one and bounded variable elimination accepts the trade. Each resolvent has the form

```
p(1,2) v p(1,3) v ... v p(1,10) v ~p(j,1)
```

Thirty literals in eleven clauses become one hundred literals in ten. The binary clause `~p(1,1) v ~p(j,1)` touched two variables; its replacement touches ten, and couples pigeon 1's entire row to pigeon j.

That is exactly the object the paper's own Discussion identifies as fatal: "a learned clause that spans two otherwise independent components merges them,

destroying the decomposition the cache depends on." A BVE resolvent spans components for the same reason and does the same damage. The paper states the principle and fails to apply it to the mechanism it shipped.

The diagnostic confirming this is that the hit rate barely moves, 49.5% against 48.5%. The cache recognises repeats equally well in both configurations. What changes is the number of distinct components reachable at all, 4,097 against 582,908. Preprocessing did not break the cache. It multiplied the universe the cache has to cover.

The general statement: bounded variable elimination's acceptance criterion is clause count, which is the right proxy for a watched-literal CDCL solver and blind to the quantity a decomposing solver depends on, namely the connectivity of the variable interaction graph. A preprocessor tuned for one architecture is contraindicated for the other.

How much of this do I believe

The php result and its mechanism: high confidence. The ablation numbers reproduce from the committed binary, the encoding was verified directly from the CNF file, the resolvent arithmetic is exact, and the eliminated variable count matches the clause delta. The unchanged hit rate rules out the competing explanation that preprocessing degraded the cache itself.

The closed-form critique of Table III: high confidence. The formula recovers the true bucket counts to under one percent across four orders of magnitude of table size. This is arithmetic rather than an interpretation.

The Java 8 prior art: fairly high confidence. The treeification threshold, the red-black structure, and the hash-primary ordering are recalled from memory rather than checked against the source, so verify before publishing. The broader point, that tree buckets are established engineering practice rather than a research question, does not depend on those details.

The GANAK framing: moderate confidence. Hash-only component caching is certainly standard in modern counters, and GANAK is explicit about the resulting error probability. Whether anyone has published the hash-as-ordering-key argument for exact caches is much less certain. Treat "not seen stated" as weaker than "not stated".

The strongest caveat: none of this is new to the counting community. That preprocessing for model counting must differ from preprocessing for SAT is established; the B+E preprocessor exists for that reason. The php result is a rediscovery. Its value is that it is sharp, quantified, and sitting in this project's own data, unrecognised.

What the paper should have said

The bucket policy question is not a research question and the measurement did not answer one. Cut it to an implementation note.

The finding is the interference between the solver's two dynamic programming mechanisms. Resolution-based elimination and component caching both run over the same formula, they pull in opposite directions, and which one wins is a property of the instance family. Dubois is where elimination does everything. Pigeonhole is where elimination destroys what caching needs, at a cost of 140x and two solved instances. That is one sentence with two supporting families and a verified mechanism, more than the paper currently claims and considerably more defensible.

The obvious next experiment is a width-aware acceptance rule for elimination: reject a resolvent that increases the variable interaction graph's connectivity, even when it reduces the clause count. On pigeonhole that rule declines every elimination BVE currently accepts.